

数据库ER图

```
1  @startuml M1寄存器信息数据库设计
2  skinparam linetype ortho
3  entity permission {
4      * permission_name : text
5  }
6
7  entity register {
8      * address : integer
9      --
10     display_name : text
11     rw : text
12     default: integer
13     register_description : text
14     permission_name : text <<fk>>
15 }
16
17 entity module {
18     * module_name : text
19     --
20     module_description : text
21     base_addr : integer
22     permission_name : text <<fk>>
23 }
24
25 entity module_base_addr {
26     * module_name : text <<fk>>
27     * camera_id : integer <<fk>>
28     --
29     base_addr : integer
30 }
31
32 entity camera {
33     * camera_id : integer
34     --
35     permission_name : text <<fk>>
36 }
37
38 entity register_bits {
39     * register_addr : integer <<fk>>
40     * bit_mask : integer
41     --
42     register_bits_name : text
43     combined_id : integer <<fk>>
44     combined_bit_mask : integer
45     rw : text
46     default: integer
47     register_bits_description : text
48     permission_name : text <<fk>>
49 }
50
```

```

51 entity combined_register {
52     * combined_id : integer <<generated>>
53     * module_name : text <<fk>>
54     * camera_id : integer <<fk>>
55     --
56     combined_name : text
57     combined_description : text
58     permission_name : text <<fk>>
59 }
60
61 camera }o-down-|| combined_register
62 combined_register }o-right-|| module
63 register ||-up-||{ register_bits
64 register_bits }|-right-|| combined_register
65 module }o-up-|| module_base_addr
66 module_base_addr }o-left-|| camera
67 @enduml

```

数据库表结构

```

1  CREATE TABLE permission (
2      permission_name TEXT PRIMARY KEY
3  );
4  CREATE TABLE register (
5      address INTEGER PRIMARY KEY,
6      display_name TEXT NOT NULL,
7      rw TEXT NOT NULL,
8      defaultValue INTEGER NOT NULL DEFAULT 0,
9      register_description TEXT NOT NULL,
10     permission_name TEXT NOT NULL,
11     FOREIGN KEY (permission_name) REFERENCES permission (permission_name)
12 );
13 CREATE TABLE module (
14     module_name TEXT PRIMARY KEY,
15     module_description TEXT NOT NULL,
16     permission_name TEXT NOT NULL,
17     FOREIGN KEY (permission_name) REFERENCES permission (permission_name)
18 );
19 CREATE TABLE module_base_addr (
20     module_name TEXT,
21     camera_id INTEGER,
22     base_addr INTEGER NOT NULL,
23     PRIMARY KEY (module_name, camera_id),
24     FOREIGN KEY (module_name) REFERENCES module (module_name),
25     FOREIGN KEY (camera_id) REFERENCES camera (camera_id)
26 );
27 CREATE TABLE camera (
28     camera_id INTEGER PRIMARY KEY,
29     permission_name TEXT NOT NULL,
30     FOREIGN KEY (permission_name) REFERENCES permission (permission_name)
31 );
32 CREATE TABLE combined_register (

```

```
33 combined_id INTEGER PRIMARY KEY AUTOINCREMENT,
34 module_name TEXT,
35 camera_id INTEGER,
36 combined_name TEXT,
37 display_name TEXT,
38 combined_description TEXT,
39 permission_name TEXT,
40 FOREIGN KEY (module_name) REFERENCES module (module_name),
41 FOREIGN KEY (camera_id) REFERENCES camera (camera_id),
42 FOREIGN KEY (permission_name) REFERENCES permission (permission_name)
43 );
44 CREATE TABLE register_bits (
45     register_addr INTEGER,
46     bit_mask INTEGER,
47     register_bits_name TEXT,
48     combined_id INTEGER,
49     combined_bit_mask INTEGER,
50     rw TEXT,
51     default INTEGER,
52     register_bits_description TEXT,
53     permission_name TEXT,
54     FOREIGN KEY (register_addr) REFERENCES register (address),
55     FOREIGN KEY (combined_id) REFERENCES combined_register (combined_id),
56     FOREIGN KEY (permission_name) REFERENCES permission (permission_name)
57 );
```

数据库表数据示例

- permission

permission_name
Develop
A
B
C

详细描述如下：

键	含义	取值
permission_name	权限种类	Administrator（管理员级别权限）

- register

address	display_name	rw	default	register_description	permission_name
0x00005000	s_hist_0[7:0]	RO	0xFF		Develop

address	display_name	rw	default	register_description	permission_name
0x00000D3A	r_nDGainMax[7:0]	RW	0x28	0~16383	Develop
0x00000D3B	r_nDGainMax[15:8]	RW	0x00		Develop
0x0000213A	r_nDGainMax[7:0]	RW	0x28	0~16383	Develop
0x0000213B	r_nDGainMax[15:8]	RW	0x00		Develop
0x00005100	xxxxxxx	RW	0x87		Develop

详细描述如下：

键	含义	取值
address	寄存器地址（主键）	32bit整型
display_name		string
rw	可读/写	'RW'（读写）、'RO'（只读）
default_value	默认值	0-255
register_description	寄存器描述，一般存放取值范围等（没有限制功能）	string
permission_name	权限（外键，引用自表permission）	'Administrator'

- module

module_name	module_description	permission_name
CAECStatistic_V60_V		Develop
AEC		Develop
CYUVDNS_V60_HW		Develop

详细描述如下：

键	含义	取值
module_name	模块名称，由CSV决定（主键）	string
module_description	模块描述	string
permission_name	权限（外键，引用自表permission）	'Administrator'

- module_base_addr

module_name	camera_id	base_addr
CAECStatistic_V60_V	NULL	0x00005000
AEC	0	0x00000BE0
AEC	1	0x00001FE0
CYUVDNS_V60_HW	NULL	0x00005100

详细描述如下：

键	含义	取值
module_name	模块名称，由CSV决定（外键，引用自表module，组合主键）	string
camera_id	相机ID（外键，引用自表camera，组合主键）	0-1
base_addr	该模块的基地址，存放该模块的寄存器的集合	32bit整型

- camera

camera_id	permission_name
0	Develop
1	Develop

详细描述如下：

键	含义	取值
camera_id	相机ID（主键）	0-1
permission_name	权限（外键，引用自表permission）	'Administrator'

- combined_register

combined_id	module_name	camera_id	combined_name	combined_description	permission_name
0	CAECStatistic_V60_V	NULL	s_hist_0	Single register	Develop
1	AEC	0	r_nDGainMax	Combined by two registers	Develop
2	AEC	1	r_nDGainMax	Combined by two registers	Develop
3	CYUVDNS_V60_HW	NULL	r_dpc_en	Split register	Develop
4	CYUVDNS_V60_HW	NULL	r_dns_en	Split register	Develop

详细描述如下：

键	含义	取值
combined_id	组合寄存器ID（主键，自增）	非负整型
module_name	模块名称，由CSV决定（外键，引用自表module）	string
camera_id	相机ID（外键，引用自表camera）	0-1
combined_name	寄存器名，用于定位combined_id	string
display_name	寄存器的含义	string
combined_description	描述	string
permission_name	权限（外键，引用自表permission）	'Administrator'

- register_bit_info

register_addr	register_bits_name	bit_mask	combined_id	combined_bit_mask	rw	default	register_bits_description	permission_name
0x00005000	CAECStatistic_V60_V_s_hist_0[7:0]	0xFF	0	0x000000FF	RO	0xFF		Develop
0x00000D3A	cam0_AEC_r_nDGainMax[7:0]	0xFF	1	0x000000FF	RW	0x28	0-16383	Develop
0x00000D3B	cam0_AEC_r_nDGainMax[15:8]	0xFF	1	0x0000FF00	RW	0x00		Develop
0x0000213A	cam1_AEC_r_nDGainMax[7:0]	0xFF	2	0x000000FF	RW	0x28	0-16383	Develop
0x0000213B	cam1_AEC_r_nDGainMax[15:8]	0xFF	2	0x0000FF00	RW	0x00		Develop
0x00005100	CYUVDNS_V60_HW_r_dpc_en	0x01	3	0x00000000	RW	0x01		Develop
0x00005100	CYUVDNS_V60_HW_r_dns_en	0x02	4	0x00000000	RW	0x02		Develop

详细描述如下：

键	含义	取值
register_addr	寄存器地址（组合主键）	32bit整型
bit_mask	寄存器有效位（组合主键）	8bit整型
register_bits_name	寄存器位名称	string
combined_id	组合寄存器ID	整型
combined_bit_mask	寄存器有效位在组合寄存器的位信息，用于拼接	整型
rw	可读/写	'RW'（读写）、'RO'（只读）
default_value	默认值	0-255
register_bit_description	寄存器描述，一般存放取值范围等（没有限制功能）	string
permission_name	权限（外键，引用自表permission）	'Administrator'

- awb_calibration_register

awb_register_name	module_name	combined_name
0	AWB_STATIS_V60_A	r_nor_tab_bor[0]
1	AWB_STATIS_V60_A	r_nor_tab_bor[1]
2	AWB_STATIS_V60_A	r_div_norm_step[0]
3	AWB_STATIS_V60_A	r_div_norm_step[1]

详细描述如下：

键	含义	取值
awb_register_name	AWB寄存器（主键）	整型
module_name	模块名称	string
combined_name	组合寄存器名	string

- ui_module

module_name	permission_name
ImgFmt	Administrator
AEC	Administrator

详细描述如下：

键	含义	取值
module_name	模块名称（主键，由界面决定，不是CSV）	string
permission_name	权限（外键，引用自表permission）	'Administrator'

- ui_info

module_name	control_name	control_index	combined_name	register_address	permission_name
AEC	BandEnable L	0	r_pBandEnable[0]	NULL	Administrator
AEC	BandEnable M	0	r_pBandEnable[1]	NULL	Administrator
AEC	OneBandLim L	0	r_pOneBandLimEnable[0]	NULL	Administrator
AEC	OneBandLim M	0	r_pOneBandLimEnable[1]	NULL	Administrator
AEC	Enable	0	r_bGradedExpoLimEn	NULL	Administrator

详细描述如下：

键	含义	取值
module_name	模块名称（组合主键）	string
control_name	控件名称（组合主键）	string
control_index	控件索引（组合主键）	整型
combined_name	寄存器名，用于定位combined_id	string
register_address	寄存器地址	
permission_name	权限（外键，引用自表permission）	'Administrator'

- ui_info_regmodule

ui_module_name	ui_control_name	reg_module_name	has_camera_id
AEC	BandEnable L	AEC	1
AEC	BandEnable M	AEC	1
AEC	OneBandLim L	AEC	1
AEC	OneBandLim M	AEC	1

详细描述如下：

键	含义	取值
module_name	模块名称	string
control_name	控件名称	string
reg_module_name	寄存器模块名称（外键，引用自表module）	string
has_camera_id	是否存在camera id	0-1

ORM接口

```
1 class Permission {
2     public:
3         static vector<Permission> getAllPermissions();
4 };
5
6 class Register {
7     public:
8         int getAddress();
9         string getRegisterName();
10        string getDisplayName();
```



```

11     string getRW();
12     int getDefault();
13     string getRegisterDescription();
14     string getPermissionName();
15     vector<RegisterBits> getRegisterBits();
16     static vector<Register> getAllRegisters();
17     static Register getRegisterByAddress(int address);
18     static vector<Register> getRegistersByCameraId(int cameraId);
19     static vector<Register> getRegistersByModuleName(string moduleName);
20     static vector<Register> getRegistersByModuleNameAndCameraId(string moduleName, int
cameraId);
21 private:
22     int address;
23     string registerName;
24     string displayName;
25     string rw;
26     int default;
27     string registerDescription;
28     string permissionName;
29 };
30
31 class Module {
32 public:
33     string getModuleName();
34     string getModuleDescription();
35     string getPermissionName();
36     vector<int> getBaseAddr();
37     int getBaseAddrByCameraId(int cameraId);
38     vector<Register> getRegisters();
39     vector<Register> getRegistersByCameraId(int cameraId);
40     vector<CombinedRegister> getCombinedRegisters();
41     vector<CombinedRegister> getCombinedRegistersByCameraId(int cameraId);
42     vector<RegisterBits> getRegisterBits();
43     vector<RegisterBits> getRegisterBitsByCameraId(int cameraId);
44     static Module getModuleByModuleName(string moduleName);
45 private:
46     string moduleName;
47     string moduleDescription;
48     string permissionName;
49 };
50
51 class CombinedRegister {
52 public:
53     int getCombinedId();
54     string getModuleName();
55     int getCameraId();
56     string getCombinedName();
57     string getCombinedDescription();
58     string getPermissionName();
59     vector<RegisterBits> getRegisterBits();
60     vector<Register> getRegisters();
61     Module getModule();
62     static CombinedRegister getCombinedRegisterByCombinedId(int combinedId);

```

```

63     static CombinedRegister
getCombinedRegisterByModuleNameAndCameraIdAndCombinedName(string moduleName, int
cameraId, string combinedName);
64     static vector<CombinedRegister> getCombinedRegistersByModuleName(string
moduleName);
65     static vector<CombinedRegister> getCombinedRegistersByCameraId(int cameraId);
66     static vector<CombinedRegister> getCombinedRegistersByModuleNameAndCameraId(string
moduleName, int cameraId);
67 private:
68     int combinedId;
69     string moduleName;
70     int cameraId;
71     string combinedName;
72     string combinedDescription;
73     string permissionName;
74 };
75
76 class RegisterBits {
77 public:
78     int getRegisterAddr();
79     int getBitMask();
80     string getRegisterBitsName();
81     int getCombinedId();
82     int getCombinedBitMask();
83     string getRW();
84     int getDefault();
85     string getRegisterBitsDescription();
86     string getPermissionName();
87     Register getRegister();
88     CombinedRegister getCombinedRegister();
89     static RegisterBits getRegisterBitsByRegisterBitsName(string registerBitsName);
90     static vector<RegisterBits> getRegisterBitsByRegisterAddr(int registerAddr);
91     static vector<RegisterBits> getRegisterBitsByCombinedId(int combinedId);
92     static RegisterBits getRegisterBitsByRegisterAddrAndBitMask(int registerAddr, int
bitMask);
93 private:
94     int registerAddr;
95     int bitMask;
96     string registerBitsName;
97     int combinedId;
98     int combinedBitMask;
99     string rw;
100    int default;
101    string registerBitsDescription;
102    string permissionName;
103 };

```

